PM SHRI JNV PALNADU
Information Practices(065)
XII IP PRACTICAL 2026-27

DATA HANDLING

SERIES

Sno	INDEX	Page. No
1*	**Write a Python program to create a Pandas Series using numpy array,dictionary,scalar values**	
2*	**Write a Python program to create a Pandas Series using Indexing and Slicing**	
3	Write a Python program to create a Pandas Series using **Attributes and Methods**	
4	Write a Python program to create a Pandas Series using **Mathematical Operations**	
5	Write a Python program to create a Pandas Series using **loc** and **iloc**	
6	Write a Python program to create a Pandas Series using **Filter conditions**	
	DATAFRAME	
7	Write a Python program to Create a Data Frame quarterly sales where each row contains the item category, item name, and expenditure. Group the rows by the category and print the total expenditure per category.	
8	Write a Python program to Create a data frame for examination result and display row labels, column labels data types of each column and the dimensions	
9	Write a Python program for creation of empty ,numpyndarrays.list of dictionaries in pandas dataframe	
10	Write a Python program for a Dictionary of Lists,DataFrame from Series,Dictionary of Series in pandas dataframe	
11	**Write a Python program for Operations on rows and columns in DataFrames**	
12	**Write a Python program for Accessing DataFrames Element through Indexing in DataFrames**	
13	**Write a Python program for Accessing DataFrames Element through Slicing in DataFrames**	
14	Write a Python program using Attributes and Methods of DataFrames	
15	Write a Python program using CSV files in Pandas DataFrames	
	USING MATPLOTLIB – DATA VISUALIZATION	
16	Given the school result data, analyses the performance of the students on diff erent parameters, e.g subject wise or class wise using LINE chart.	
17	For the Data frames created analyze, and plot appropriate charts with title and legend using BAR chart	
18	Take data of your interest from an open source (e. data.gov.in), aggregate and summarize it. Then plot it using different plotting functions of the Matplotlib library.	
19	Write a python program for plotting a HISTOGRAM chart.	
	MYSQL	
20	MySQL queries using DDL(CARD),DML(UID),DQL(Select) commands in MySQL	
21	MySQL queries using SINGLE FUNCTIONS(Numeric,String,Date)	
22	MySQL queries using AGGREGRATE FUNCTIONS(Max,Min,Sum,Avg,Count(*))	
23	MySQL queries using GROUP BY,HAVING,ORDER BY(ASC,DESC)	
24	MySQL queries using CARTESIAN PRODUCT or CROSS JOIN	
25	MySQL queries using EQUI JOIN & NATURAL JOIN	

Series

Pandas



Practical 1: Write a Python program to create a Pandas Series using **numpy array**, **dictionary**, **scalar** values

Ans:

Source Code:

```
import pandas as pd
import numpy as np
# 1. Creating Series using NumPy array
a= np.array([10, 20, 30, 40])
s1 = pd.Series(a)
print("Series from NumPy Array:")
print(s1)
# 2. Creating Series using Dictionary
data = {'a': 100, 'b': 200, 'c': 300}
s2 = pd.Series(data)
print("\nSeries from Dictionary:")
print(s2)
# 3. Creating Series using Scalar Value
s3 = pd.Series(5, index=[1, 2, 3, 4])
print("\nSeries from Scalar Value:")
print(s3)
```

Output:

```
Series from NumPy Array:
0    10
1    20
2    30
3    40
dtype: int64

Series from Dictionary:
a    100
b    200
c    300
dtype: int64

Series from Scalar Value:
1    5
2    5
3    5
4    5
dtype: int64
```

Practical 2:Write a Python program to create a Pandas Series using Indexing and Slicing

Ans:

```
import pandas as pd
data = pd.Series([10, 20, 30, 40, 50]) # Create a Series
# Display the Series
print("Original Series:")
print(data)
# Indexing examples
print("\nAccessing single elements using index:")
print("Element at index 0:", data[0])
print("Element at index 3:", data[3])
# Slicing examples
print("\nAccessing multiple elements using slicing:")
print("Elements from index 1 to 3:")
print(data[1:4])  # elements at index 1,2,3
print("\nElements from beginning to index 2:")
print(data[:3])   # first three elements
print("\nElements from index 2 to end:")
print(data[2:])   # from index 2 to end
```

Output:

```
Original Series:
0    10
1    20
2    30
3    40
4    50
dtype: int64
Accessing single elements using index:
Element at index 0: 10
Element at index 3: 40

Accessing multiple elements using slicing:
Elements from index 1 to 3:
1    20
2    30
3    40
dtype: int64
Elements from beginning to index 2:
0    10
1    20
2    30
dtype: int64
Elements from index 2 to end:
2    30
3    40
4    50
dtype: int64
```

Practical 3: Write a Python program to create a Pandas Series using Attributes and Methods

Ans:

```
import pandas as pd
# Creating a Series with a name
data = pd.Series([10, 20, 30, 40, 50],index=['A', 'B', 'C', 'D', 'E'],name='Marks')
# Display the Series
print("Series:")
print(data)
# --- Using Attributes ---
print("\nAttributes of Series:")
print("Name:", data.name)           # Returns name of the Series
print("Values:", data.values)       # Returns all values
print("Index:", data.index)         # Returns index labels
print("Data Type:", data.dtype)     # Returns data type
print("Size:", data.size)           # Returns number of elements
print("Shape:", data.shape)         # Returns shape (tuple)
print("Empty:", data.empty)         # Checks if the Series is empty
# --- Using Methods ---
print("\nMethods of Series:")
print("Head (first 3 values):")
print(data.head(3))                 # Display first 3 elements
print("\nTail (last 2 values):")
print(data.tail(2))                 # Display last 2 elements
print("\nSum of elements:", data.sum())     # Sum of all elements
print("Maximum value:", data.max())         # Maximum value
print("Mean value:", data.mean())           # Mean (average)
print("Count (non-NA values):", data.count())  # Counts valid (non-missing) values
```

Output:

```
Series:
A    10
B    20
C    30
D    40
E    50
Name: Marks, dtype: int64

Attributes of Series:
Name: Marks
Values: [10 20 30 40 50]
Index: Index(['A', 'B', 'C', 'D', 'E'], dtype='object')
Data Type: int64
Size: 5
Shape: (5,)
Empty: False

Methods of Series:
Head (first 3 values):
A    10
B    20
C    30
Name: Marks, dtype: int64

Tail (last 2 values):
D    40
E    50
Name: Marks, dtype: int64

Sum of elements: 150
Maximum value: 50
Mean value: 30.0
Count (non-NA values): 5
```

Practical 4:Write a Python program to create a Pandas Series using Mathematical Operations

Ans:

```
import pandas as pd
import numpy as np
# Creating two Series with NaN (missing) values
s1 = pd.Series([10, 20, np.nan, 40, 50])
s2 = pd.Series([5, np.nan, 15, 20, 25])
# Display the Series
print("Series 1:")
print(s1)
print("\nSeries 2:")
print(s2)
# Performing mathematical operations
print("\nAddition of two Series:")
print(s1 + s2)
print("\nSubtraction of two Series:")
print(s1 - s2)
print("\nMultiplication of two Series:")
print(s1 * s2)
print("\nDivision of two Series:")
print(s1 / s2)
```

Output:

```
Series 1:
0    10.0
1    20.0
2     NaN
3    40.0
4    50.0
dtype: float64
Series 2:
0     5.0
1     NaN
2    15.0
3    20.0
4    25.0
dtype: float64
Addition of two Series:
0    15.0
1     NaN
2     NaN
3    60.0
4    75.0
dtype: float64
Subtraction of two Series:
0     5.0
1     NaN
2     NaN
3    20.0
4    25.0
dtype: float64
Multiplication of two Series:
0      50.0
1       NaN
2       NaN
3     800.0
4    1250.0
dtype: float64
Division of two Series:
0    2.0
1    NaN
2    NaN
3    2.0
4    2.0
dtype: float64
```

Practical 5:Write a Python program to create a Pandas Series using loc and iloc

Ans:

```
import pandas as pd
import numpy as np
# Creating a Series with some NaN values
marks = pd.Series([85, np.nan, 78, 92, np.nan], index=['eng', 'ip', 'maths',
'phy', 'che'])
print("Marks in subjects (with NaN values):")
print(marks)
# Accessing elements using loc (by index label)
print("\nMarks in ip using loc:", marks.loc['ip'])  # NaN
print("Marks in che using loc:", marks.loc['che'])  # NaN
# Accessing elements using iloc (by integer position)
print("\nMarks in first subject using iloc:", marks.iloc[0]) # 85
print("Marks in second subject using iloc:", marks.iloc[1])  # NaN
# Accessing multiple elements
print("\nMarks in eng and phy using loc:")
print(marks.loc[['eng', 'phy']])
print("\nMarks in first three subjects using iloc:")
print(marks.iloc[0:3])
# Checking for NaN values
print("\nSubjects with missing marks:")
print(marks[marks.isna()])
```

Output:

```
Marks in subjects (with NaN values):
eng      85.0
ipNaN
maths    78.0
phy      92.0
cheNaN
dtype: float64

Marks in ip using loc: nan
Marks in che using loc: nan

Marks in first subject using iloc: 85.0
Marks in second subject using iloc: nan

Marks in eng and phy using loc:
eng    85.0
phy    92.0
dtype: float64

Marks in first three subjects using iloc:
eng      85.0
ip        NaN
maths    78.0
dtype: float64

Subjects with missing marks:
ip        NaN
che       NaN
dtype: float64
```

Practical 6:Write a Python program to create a Pandas Series using Filter conditions

Ans:

```
import pandas as pd
# Create a numeric Pandas Series
marks = pd.Series([85, 42, 76, 90, 33, 58, 71, 49])
print("Original Marks Series:")
print(marks)
# Filter: students scoring more than 50
passed = marks[marks > 50]
print("\nStudents scoring more than 50:")
print(passed)
# Filter: students scoring between 50 and 80
range_scores = marks[(marks >= 50) & (marks <= 80)]
print("\nStudents scoring between 50 and 80:")
print(range_scores)
```

Output:

```
Original Marks Series:
0    85
1    42
2    76
3    90
4    33
5    58
6    71
7    49
dtype: int64

Students scoring more than 50:
0    85
2    76
3    90
5    58
6    71
dtype: int64

Students scoring between 50 and 80:
2    76
5    58
6    71
dtype: int64
```

DataFrame

Pandas



Practical 7: Write a Python program to Create a Data Frame quarterly sales where each row contains the item category, item name, and expenditure. Group the rows by the category and print the total expenditure per category.

Ans:

```
import pandas as pd
data = {
    'Category': ['Electronics', 'Electronics', 'Grocery', 'Grocery', 'Stationery', 'Stationery'],
    'Item': ['Smartphone', 'Laptop', 'Rice', 'Sugar', 'Notebook', 'Pen'],
    'Expenditure': [25000, 45000, 3000, 1500, 500, 200]
}
df = pd.DataFrame(data)
# Display the original DataFrame
print("Quarterly Sales Data:")
print(df)
# Group by Category and calculate total expenditure per category
grouped = df.groupby('Category')['Expenditure'].sum()
# Display the total expenditure per category
print("\nTotal Expenditure per Category:")
print(grouped)
```

Output:

Quarterly Sales Data:

	Category	Item	Expenditure
0	Electronics	Smartphone	25000
1	Electronics	Laptop	45000
2	Grocery	Rice	3000
3	Grocery	Sugar	1500
4	Stationery	Notebook	500
5	Stationery	Pen	200

Total Expenditure per Category:

```
Category
Electronics    70000
Grocery         4500
Stationery       700
Name: Expenditure, dtype: int64
```

Practical 8:Write a Python program to Create a data frame for examination result and display row labels, column labels data types of each column and the dimensions

Ans:

```
import pandas as pd
# Create DataFrame for examination result
data = {
    'Name': ['Rahul', 'Priya', 'Amit', 'Neha', 'Sonia'],
   'Roll_No': [101, 102, 103, 104, 105],
    'Marks': [85, 92, 78, 88, 95],    'Grade': ['B', 'A', 'C', 'B', 'A']
}
df = pd.DataFrame(data)
# Display DataFrame
print("Examination Result DataFrame:\n", df)
# Display Row Labels (Index)
print("\nRow Labels (Index):", df.index)
# Display Column Labels
print("Column Labels:", df.columns)
# Display Data Types of each column
print("\nData Types:\n", df.dtypes)
# Display Dimensions of DataFrame (rows, columns)
print("\nDimensions of DataFrame:", df.shape)
```

Output:

```
Examination Result DataFrame:
Name  Roll_No  Marks Grade
0  Rahul      101     85     B
1  Priya      102     92     A
2   Amit      103     78     C
3   Neha      104     88     B
4  Sonia      105     95     A
Row Labels (Index): RangeIndex(start=0, stop=5, step=1)
Column Labels: Index(['Name', 'Roll_No', 'Marks', 'Grade'], dtype='object')
Data Types:
 Name       object
Roll_No     int64
Marks       int64
Grade      object
dtype: object

Dimensions of DataFrame: (5, 4)
```

Practical 9:Write a Python program for creation of empty ,numpy ndarrays.list of dictionaries in pandas dataframe.

Ans:

```
import pandas as pd
import numpy as np
# ---------- 1. Empty DataFrame ----------
empty_df = pd.DataFrame()
print("1. Empty DataFrame:")
print(empty_df)
# ---------- 2. DataFrame from NumPy ndarray ----------
# Create a 2D NumPy array
arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
# Convert ndarray to DataFrame
df_from_array = pd.DataFrame(arr, columns=['Math', 'Science', 'English'])
print("\n2. DataFrame from NumPy ndarray:")
print(df_from_array)

# ---------- 3. DataFrame from list of dictionaries ----------
# Create a list of dictionaries
data = [
    {'Name': 'Ravi', 'Age': 16, 'Grade': 'A'},
    {'Name': 'Anita', 'Age': 17, 'Grade': 'B'},
    {'Name': 'Suresh', 'Age': 16, 'Grade': 'A+'}
]
# Convert list of dictionaries to DataFrame
df_from_dict = pd.DataFrame(data)
print("\n3. DataFrame from list of dictionaries:")
print(df_from_dict)
```

Output:

```
1. Empty DataFrame:
Empty DataFrame
Columns: []
Index: []
2. DataFrame from NumPy ndarray:
   Math  Science  English
0    10       20       30
1    40       50       60
2    70       80       90
3. DataFrame from list of dictionaries:
    Name  Age Grade
0    Ravi   16     A
1   Anita   17     B
2  Suresh   16    A+
```

Practical 10:Write a Python program for a Dictionary of Lists, DataFrame from Series, Dictionary of Series in pandas DataFrame.

Ans:

```
import pandas as pd
print("1. DataFrame from Dictionary of Lists:")
data_lists = {
    'Name': ['Ravi', 'Anita', 'Suresh', 'Priya'], 'Marks': [85, 92, 78, 88],   'Age': [16, 17, 16, 17]
}
df_lists = pd.DataFrame(data_lists)
print(df_lists)
print("\n2. DataFrame from Series:")
# Create individual Series
names = pd.Series(['Ravi', 'Anita', 'Suresh', 'Priya'])
marks = pd.Series([85, 92, 78, 88])
age = pd.Series([16, 17, 16, 17])
# Create DataFrame by combining Series
df_series = pd.DataFrame({'Name': names, 'Marks': marks, 'Age': age})
print(df_series)
# ---------- 3. DataFrame from Dictionary of Series ----------
print("\n3. DataFrame from Dictionary of Series:")
# Create Series with custom indexes
names_s = pd.Series(['Ravi', 'Anita', 'Suresh', 'Priya'], index=[101,102,103,104])
marks_s = pd.Series([85, 92, 78, 88], index=[101,102,103,104])
age_s = pd.Series([16, 17, 16, 17], index=[101,102,103,104])
# Combine into DataFrame
df_dict_series = pd.DataFrame({'Name': names_s, 'Marks': marks_s, 'Age': age_s})
print(df_dict_series)
```

Output:

```
1. DataFrame from Dictionary of Lists:
     Name  Marks  Age
0    Ravi     85   16
1   Anita     92   17
2  Suresh     78   16
3   Priya     88   17
2. DataFrame from Series:
     Name  Marks  Age
0    Ravi     85   16
1   Anita     92   17
2  Suresh     78   16
3   Priya     88   17
3. DataFrame from Dictionary of Series:
     Name  Marks  Age
101    Ravi     85   16
102   Anita     92   17
103  Suresh     78   16
104   Priya     88   17
```

Practical 11: Write a Python program for Operations on rows and columns in DataFrames

Ans:

```
import pandas as pd
data = {  'Name': ['Ravi', 'Anita', 'Suresh', 'Priya'],'Marks': [85, 42, 76, 90], 'Age': [16, 17, 16, 18]  }
df = pd.DataFrame(data)
print("Original DataFrame:\n",df)
# Access a single column
print("\n Marks Column:\n",df['Marks'])
# Access multiple columns
print("\nName and Age Columns:\n",df[['Name', 'Age']])
#print(df[['Name', 'Age']])
# Add a new column
df['Grade'] = ['A', 'C', 'B', 'A']
print("\nDataFrame after adding Grade column:",df)
# Delete a column
df.drop('Age', axis=1, inplace=True)
print("\nDataFrame after deleting Age column:",df)
# Rename column labels
df.rename(columns={'Marks': 'Score', 'Grade': 'Result'}, inplace=True)
print("\nDataFrame after renaming columns (Marks → Score, Grade → Result):",df)
# Access a single row using index
print("\nRow at index 2:",df.iloc[2])
# Access multiple rows using slicing
print("\nRows from index 1 to 3:",df[1:3])
# Add a new row
new_row = pd.DataFrame({'Name': ['Rahul'], 'Score': [78], 'Result': ['B']})
df = pd.concat([df, new_row], ignore_index=True)
print("\nDataFrame after adding a new row:",df)
# Delete a row by index
df.drop(1, axis=0, inplace=True)
print("\nDataFrame after deleting row at index 1:",df)
# Rename row labels (index)
df.rename(index={0: 'R1', 2: 'R3', 3: 'R4', 4: 'R5'}, inplace=True)
print("\nDataFrame after renaming row labels:",df)
```

Output:

```
Original DataFrame:
   Name  Marks  Age
0   Ravi     85   16
1  Anita     42   17
2 Suresh     76   16
3  Priya     90   18
Marks Column:
0    85
1    42
2    76
3    90
Name: Marks, dtype: int64
Name and Age Columns:
   Name  Age
0   Ravi   16
1  Anita   17
2 Suresh   16
3  Priya   18
DataFrame after adding Grade column:
   Name  Marks  Age Grade
0   Ravi     85   16     A
1  Anita     42   17     C
2 Suresh     76   16     B
3  Priya     90   18     A

DataFrame after deleting Age column:
   Name  Marks Grade
0   Ravi     85     A
1  Anita     42     C
2 Suresh     76     B
3  Priya     90     A
```

DataFrame after renaming columns (Marks → Score, Grade → Result):

	Name	Score	Result
0	Ravi	85	A
1	Anita	42	C
2	Suresh	76	B
3	Priya	90	A

Row at index 2:

Name Suresh
Score 76
Result B
Name: 2, dtype: object

Rows from index 1 to 3:

	Name	Score	Result
1	Anita	42	C
2	Suresh	76	B

DataFrame after adding a new row:

	Name	Score	Result
0	Ravi	85	A
1	Anita	42	C
2	Suresh	76	B
3	Priya	90	A
4	Rahul	78	B

DataFrame after deleting row at index 1:

	Name	Score	Result
0	Ravi	85	A
2	Suresh	76	B
3	Priya	90	A
4	Rahul	78	B

DataFrame after renaming row labels:

	Name	Score	Result
R1	Ravi	85	A
R3	Suresh	76	B
R4	Priya	90	A
R5	Rahul	78	B

Practical 12:Write a Python program for Accessing DataFrames Element through Indexing in DataFrames

Ans:

```
import pandas as pd
# Create a sample DataFrame
data = {
    "Name": ["Ravi", "Anita", "Suresh", "Priya", "Rahul"],    "Marks": [85, 42, 76, 90, 33],
"Age": [16, 17, 16, 17, 16]
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
# ---------- Accessing using label-based indexing (loc) ----------
# Access a single row by label
print("\nRow with index 2 (using loc):")
print(df.loc[2])
# Access multiple rows and columns by labels
print("\nRows 1 to 3, columns 'Name' and 'Marks' (using loc):")
print(df.loc[1:3, ["Name", "Marks"]])
# ---------- Accessing using position-based indexing (iloc) ----------
# Access a single element (row 0, column 1)
print("\nElement at row 0, column 1 (using iloc):")
print(df.iloc[0, 1])
# Access multiple rows and columns by positions
print("\nRows 0 to 2, columns 0 and 2 (using iloc):")
print(df.iloc[0:3, [0, 2]])
# ---------- Boolean indexing ----------
# Filter students who scored more than 50
print("\nStudents scoring more than 50 (Boolean indexing):")
print(df[df["Marks"] > 50])
# Filter students with Marks > 50 and Age = 16
print("\nStudents with Marks > 50 and Age = 16:")
print(df[(df["Marks"] > 50) & (df["Age"] == 16)])
```

Output:

```
Original DataFrame:
   Name  Marks  Age
0   Ravi     85   16
1  Anita     42   17
2 Suresh     76   16
3  Priya     90   17
4  Rahul     33   16

Row with index 2 (using loc):
Name     Suresh
Marks        76
Age          16
Name: 2, dtype: object

Rows 1 to 3, columns 'Name' and 'Marks' (using loc):
   Name  Marks
1  Anita     42
2 Suresh     76
3  Priya     90

Element at row 0, column 1 (using iloc):
85
Rows 0 to 2, columns 0 and 2 (using iloc):
   Name  Age
0   Ravi   16
1  Anita   17
2 Suresh   16
Students scoring more than 50 (Boolean indexing):
   Name  Marks  Age
0   Ravi     85   16
2 Suresh     76   16
3  Priya     90   17
Students with Marks > 50 and Age = 16:
   Name  Marks  Age
0   Ravi     85   16
2 Suresh     76   16
```

Practical 13:Write a Python program for Accessing DataFrames Element through Slicing in DataFrames

Ans:

```
import pandas as pd

# Create a DataFrame

data = {

    'Name': ['Amit', 'Riya', 'Karan', 'Neha', 'Tina'], 'Marks': [85, 90, 78, 92, 88],

    'Grade': ['B', 'A', 'C', 'A', 'B']

}

df = pd.DataFrame(data, index=['a', 'b', 'c', 'd', 'e'])

# Display the DataFrame

print("Original DataFrame:")

print(df)

# Accessing rows using loc (label-based slicing)

print("\nUsingloc - Rows from label 'b' to 'd':")

print(df.loc['b':'d'])

# Accessing rows using iloc (integer-based slicing)

print("\nUsingiloc - Rows from index 1 to 3:")

print(df.iloc[1:4])

# Accessing specific rows and columns using loc

print("\nUsingloc - Names and Marks from 'b' to 'd':")

print(df.loc['b':'d', ['Name', 'Marks']])

# Accessing specific rows and columns using iloc

print("\nUsingiloc - First 3 rows and first 2 columns:")

print(df.iloc[0:3, 0:2])
```

Output:

Original DataFrame:

```
   Name  Marks Grade
a Amit     85     B
b Riya     90     A
c  Karan  78     C
d  Neha   92     A
e   Tina   88     B
```

Using loc - Rows from label 'b' to 'd':

```
   Name  Marks Grade
B Riya     90        A
c  Karan   78      C
dNeha      92      A
```

Using iloc - Rows from index 1 to 3:

```
   Name  Marks Grade
b  Riya     90     A
c  Karan   78     C
d  Neha    92     A
```

Using loc - Names and Marks from 'b' to 'd':

```
   Name  Marks
b Riya     90
c  Karan  78
d  Neha     92
```

Using iloc - First 3 rows and first 2 columns:

```
   Name  Marks
a Amit     85
b Riya     90
c  Karan     78
```

Practical 14:Write a Python program using Attributes and Methods of DataFrames

Ans:

```
import pandas as pd

# Create a simple DataFrame

data = {

    'Name': ['Amit', 'Riya', 'Karan', 'Neha'],    'Age': [17, 18, 16, 17],  'Marks': [88, 92, 79, 85]

}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)
print()
 #---------------- ATTRIBUTES ----------------

print("ATTRIBUTES OF DATAFRAME:")

print("Index   :", df.index)        # Row index labels

print("Columns :", df.columns)      # Column labels

print("Data Types:\n", df.dtypes)   # Data type of each column

print("Values  :\n", df.values)     # All data as 2D array

print("Shape   :", df.shape)        # (rows, columns)

print("Size    :", df.size)         # Total number of elements

print("Transpose (df.T):\n", df.T)  # Transposed DataFrame

print("Number of Dimensions:", df.ndim)   # Dimension of DataFrame

print("Is DataFrame Empty?:", df.empty)   # Check if DataFrame is empty

print()

# ---------------- METHODS ----------------
print("METHODS OF DATAFRAME:")

print("Head (first 2 rows):\n", df.head(2))    # Display first 2 rows

print("Tail (last 2 rows):\n", df.tail(2))     # Display last 2 rows

print("Length of DataFrame using len():", len(df))  # Number of rows

print("Count of Non-Null Values in Each Column:\n", df.count())  # Count non-null

print("Axes of DataFrame:", df.axes)           # Row and column index objects
```

Output:

```
Original DataFrame:
    Name  Age  Marks
0   Amit   17     88
1   Riya   18     92
2  Karan   16     79
3   Neha   17     85

ATTRIBUTES OF DATAFRAME:
Index   : RangeIndex(start=0, stop=4, step=1)
Columns : Index(['Name', 'Age', 'Marks'], dtype='object')
Data Types:
 Name     object
Age       int64
Marks     int64
dtype: object
Values  :
 [['Amit' 17 88]
 ['Riya' 18 92]
 ['Karan' 16 79]
 ['Neha' 17 85]]
Shape   : (4, 3)
Size    : 12
Transpose (df.T):
          0        1      2        3
Name   Amit Riya  Karan  Neha
Age      17      18     16    17
Marks  88      92     79    85
Number of Dimensions: 2
Is DataFrame Empty?: False

METHODS OF DATAFRAME:
Head (first 2 rows):
   Name  Age  Marks
0  Amit   17     88
1  Riya   18     92
Tail (last 2 rows):
    Name  Age  Marks
2  Karan   16     79
3   Neha   17     85
Length of DataFrame using len(): 4
Count of Non-Null Values in Each Column:
 Name     4
Age      4
Marks    4
dtype: int64
Axes of DataFrame: [RangeIndex(start=0, stop=4, step=1), Index(['Name', 'Age', 'Marks'],
dtype='object')]
```

Practical 15:Write a Python program using CSV files in Pandas DataFrames

Ans:

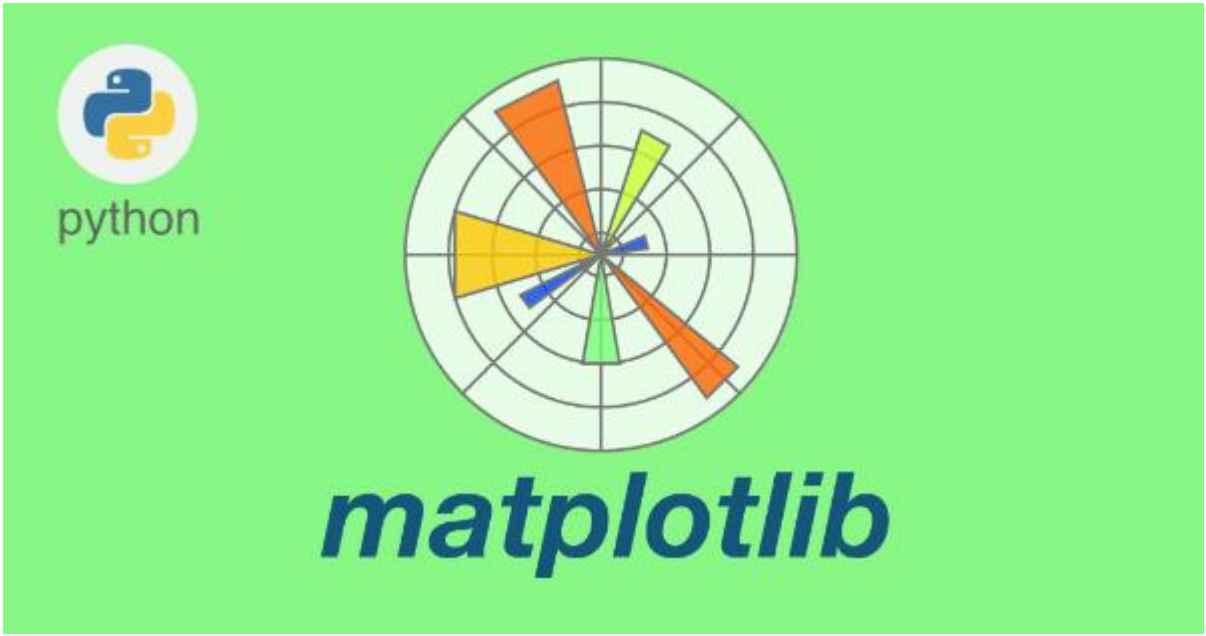
```
import os
print(os.getcwd())
import pandas as pd
# ---------------- EXPORTING DATAFRAME TO CSV ----------------
# Create a simple DataFrame
data = {
    'Name': ['Amit', 'Riya', 'Karan', 'Neha'],
    'Age': [17, 18, 16, 17],
    'Marks': [88, 92, 79, 85]
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
# Export the DataFrame to a CSV file
df.to_csv("students.csv", index=False)
print("\nDataFrame has been successfully exported to 'students.csv'.")
print()
# ---------------- IMPORTING DATAFRAME FROM CSV ----------------
# Read the CSV file into a new DataFrame
new_df = pd.read_csv("students.csv")
print("Data imported from CSV file:")
print(new_df)
```

Output:

```
Original DataFrame:
Name  Age  Marks
0   Amit   17     88
1   Riya   18     92
2  Karan   16     79
3   Neha   17     85

DataFrame has been successfully exported to 'students.csv'.

Data imported from CSV file:
Name  Age  Marks
0   Amit   17     88
1   Riya   18     92
2  Karan   16     79
3   Neha   17     85
```

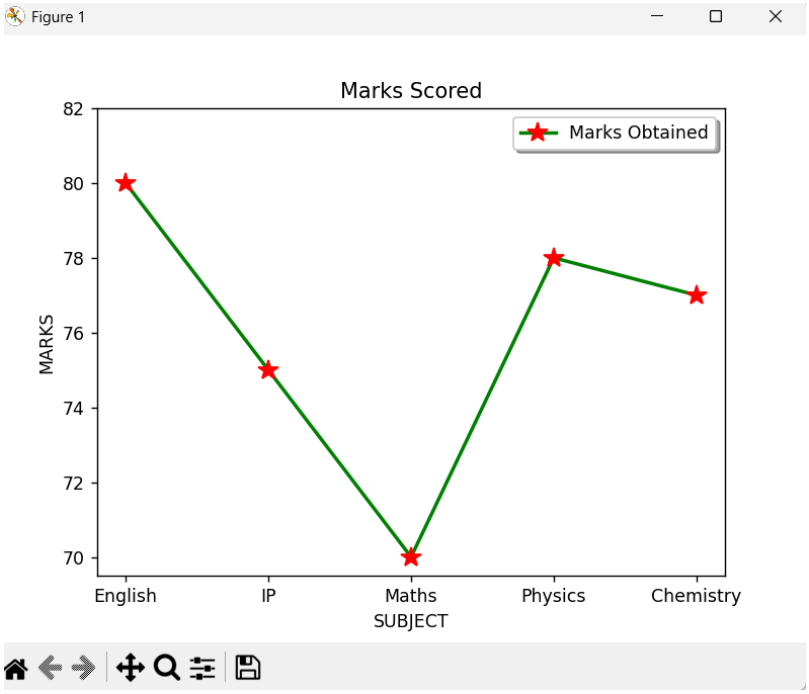

python
matplotlib

Practical 16:Given the school result data, analyses the performance of the students on different parameters, e.g subject wise or class wise using LINE chart.

Ans:

```
import pandas as pd
import matplotlib.pyplot as plt
# Subjects and corresponding marks
subject = ['English', 'IP', 'Maths', 'Physics', 'Chemistry']
marks = [80, 75, 70, 78, 77]
# Plotting the line chart with legend
plt.plot(subject, marks,
color='green',          # Line color
marker='*',             # Star marker
markerfacecolor='red',  # Fill color for marker
markeredgecolor='red',  # Border color for marker
markersize=12,          # Size of marker
linewidth=2,            # Thickness of the line
label='Marks Obtained') # Legend label
# Adding title and axis labels
plt.title('Marks Scored')
plt.xlabel('SUBJECT')
plt.ylabel('MARKS')
# Adding custom ticks
plt.xticks(subject)
plt.yticks([70, 72, 74, 76, 78, 80, 82])
# Adding legend (placed at best location automatically)
plt.legend(loc='best', fontsize=10, frameon=True, shadow=True)
# Save the chart as an image
plt.savefig("d:\\line.png")
# Display the chart
plt.show()
```

Output:



Practical 17:For the Data frames created analyze, and plot appropriate charts with title and legend using BAR chart

Ans:

```
import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_csv("D:\\Mela3.csv")
#creates plot of different color for each week
df.plot(kind='bar',x='Day',title='Mela Sales Report',
color=['red','yellow','purple'],
edgecolor='green',linewidth=2,linestyle='--')
plt.title('Mela Sales Report')
plt.xlabel('Days')
plt.ylabel('Sales in Rs')
plt.savefig("d:\\ex.png")
plt.show()
```

Output:



Practical 18:Take data of your interest from an open source (e. data.gov.in), aggregate and summarize it. Then plot it using different plotting functions of the Matplotlib library.

Ans:

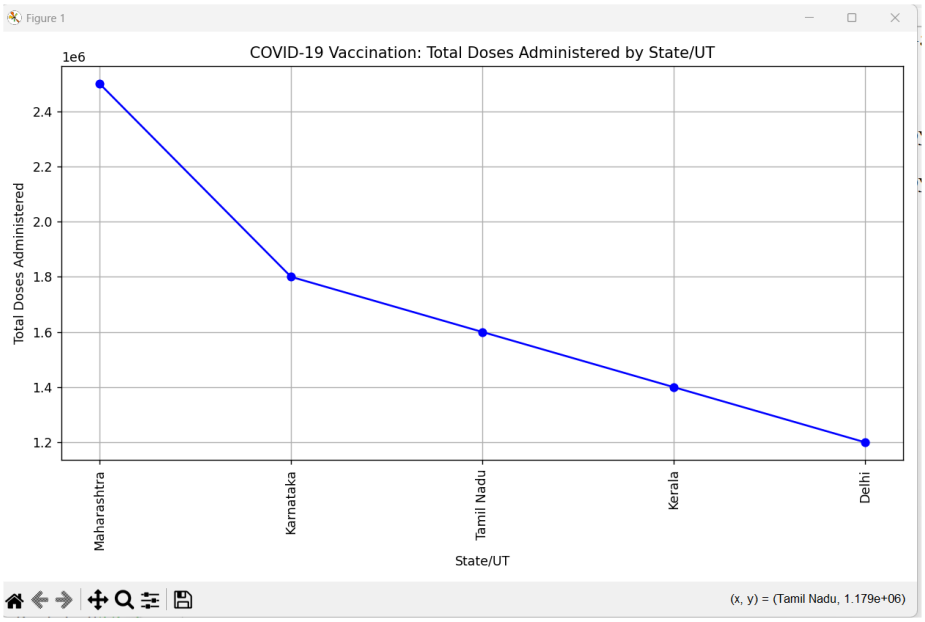
First create CSV files below

State/UT,Total Doses Administered,People Vaccinated
Delhi,1200000,600000
Maharashtra,2500000,1250000
Karnataka,1800000,900000
Tamil Nadu,1600000,800000
Kerala,1400000,700000
And save as ex.csv

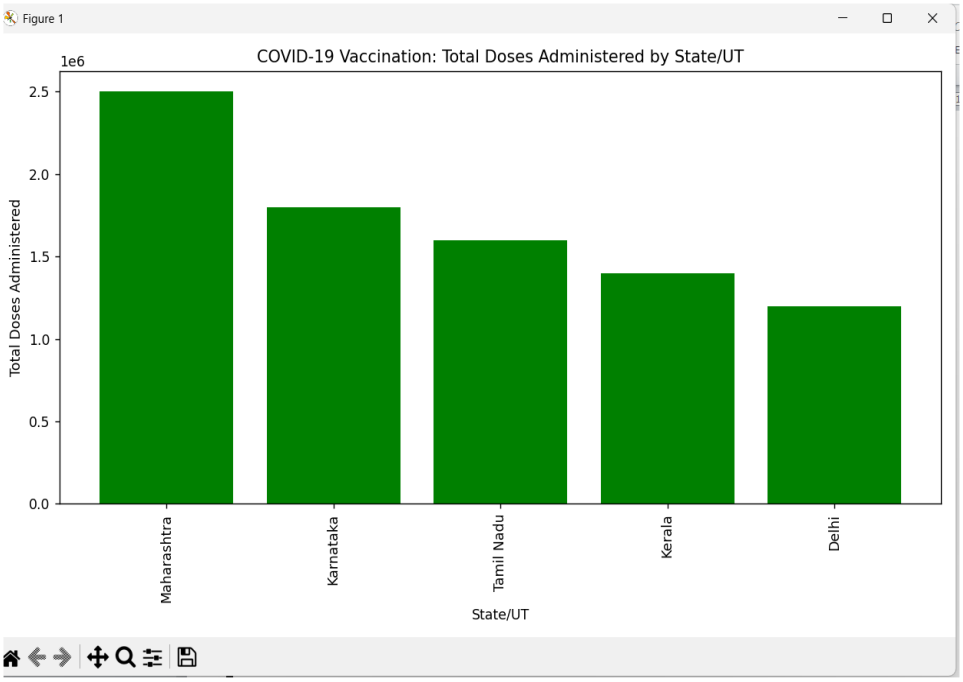
```
import pandas as pd
import matplotlib.pyplot as plt
# Step 1: Load the dataset
df = pd.read_csv('d:\\ex.csv')
# Step 2: Data Preprocessing
# Assuming the dataset has columns 'State/UT', 'Total Doses Administered', and 'People Vaccinated'
df = df[['State/UT', 'Total Doses Administered', 'People Vaccinated']]
# Step 3: Aggregation
# Sorting the data by 'Total Doses Administered'
df_sorted = df.sort_values(by='Total Doses Administered', ascending=False)
# Step 4: Visualization
# Line Plot
plt.figure(figsize=(10, 6))
plt.plot(df_sorted['State/UT'], df_sorted['Total Doses Administered'], marker='o', linestyle='-',
color='b')
plt.title('COVID-19 Vaccination: Total Doses Administered by State/UT')
plt.xlabel('State/UT')
plt.ylabel('Total Doses Administered')
plt.xticks(rotation=90)
plt.grid(True)
plt.tight_layout()
plt.show()
# Bar Chart
plt.figure(figsize=(10, 6))
plt.bar(df_sorted['State/UT'], df_sorted['Total Doses Administered'], color='g')
plt.title('COVID-19 Vaccination: Total Doses Administered by State/UT')
plt.xlabel('State/UT')
plt.ylabel('Total Doses Administered')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
# Histogram
plt.figure(figsize=(10, 6))
plt.hist(df_sorted['Total Doses Administered'], bins=10, color='r', edgecolor='black')
plt.title('Distribution of Total Doses Administered')
plt.xlabel('Total Doses Administered')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```

Output: It will display all 3 types of charts Line,Bar,Histogram graph

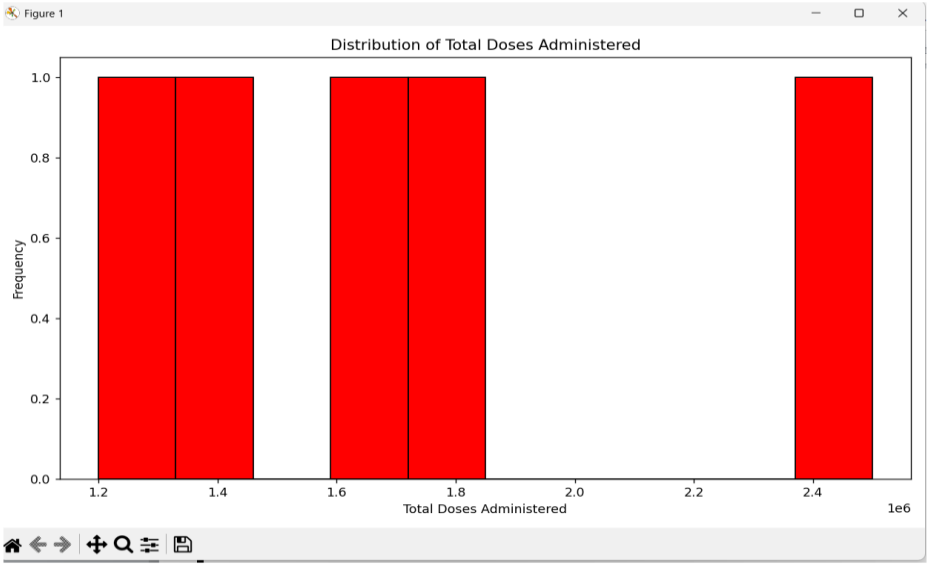
LINE CHART



BAR CHART



HISTOGRAM

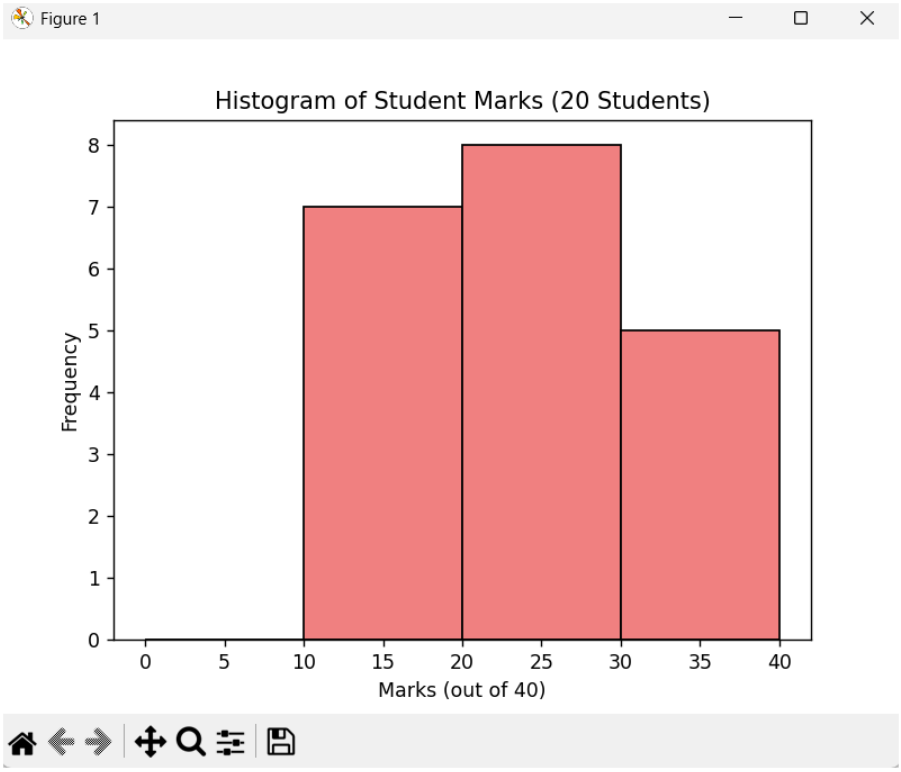


Practical 19: Write a python program for plotting a HISTOGRAM chart.

Output:

```
import matplotlib.pyplot as plt
# Marks of 20 students in a test (out of 40)
marks = [12, 25, 18, 30, 22, 35, 28, 14, 10, 26,
         32, 19, 15, 38, 21, 17, 24, 29, 27, 36]
# Plot Histogram (Normal frequency distribution)
plt.hist(
marks,
bins=[0, 10, 20, 30, 40],   # class intervals (0–10,10–20,20–30,30–40)
cumulative=False,           # normal frequency (not cumulative)
histtype='bar',             # default bar style
align='mid',                # bins aligned at mid
orientation='vertical',     # vertical histogram
color='lightcoral', edgecolor='black'
)
# Add labels and title
plt.xlabel("Marks (out of 40)")
plt.ylabel("Frequency")
plt.title("Histogram of Student Marks (20 Students)")
# Save figure as PNG file
plt.savefig("D:\\H.png", dpi=300)
# Show plot
plt.show()
```

Output:




MySQL®

Practical 20:MySQL queries using DDL(CARD),DML(UID),DQL(Select) commands in MySQL

(a) Write a Query to Create a new database in the name of **XII**

```
mysql> create database XII;
Query OK, 1 row affected (0.02 sec)
```

(b) Write a Query to Open the database **XII**.

```
mysql> use XII;
Database changed
```

(c) Write a Query to list all the existing database names.**SHOW DATABASES**;

```
+--------------------+
| Database           |
+--------------------+
| employees          |
| hospital           |
| hospitals          |
| information_schema |
| ip_practicals      |
| mysql              |
| performance_schema |
| school             |
| students           |
| sys                |
+--------------------+
```

(d) Write a Query to create the table called: **Student**

```
mysql> CREATE TABLE student (
    ->       student_id INT PRIMARY KEY,
    ->       name VARCHAR(50),
    ->       marks INT
    -> );
Query OK, 0 rows affected (0.29 sec)
```

(e) Write a Query to List all the tables that exists in the current database.

```
mysql> show tables;
+----------------+
| Tables_in_xii  |
+----------------+
| student        |
+----------------+
1 row in set (0.01 sec)
```

(f)Write a Query to insert all the rows of into student table

```
mysql> INSERT INTO student (student_id, name, marks) VALUES
    -> (1, 'Rahul', 85),
    -> (2, 'Priya', 92),
    -> (3, 'Amit', 78),
    -> (4, 'Sneha', 88),
    -> (5, 'Karan', 95);
Query OK, 5 rows affected (0.06 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

(g)Write a Query to display all the details of the Employees from the table '**STUDENT**'.

```
mysql> select*from student;
+------------+--------+-------+
| student_id | name   | marks |
+------------+--------+-------+
|          1 | Rahul  |    85 |
|          2 | Priya  |    92 |
|          3 | Amit   |    78 |
|          4 | Sneha  |    88 |
|          5 | Karan  |    95 |
+------------+--------+-------+
5 rows in set (0.01 sec)
```

(h)Write a Query tomodify the structure of the table using **Alter** command

```
mysql> Alter table student add dob date;
```

(i) Write a Query to update the values in student table for DOB using **UPDATE** command

```
mysql> update student set dob='2025-10-10' where name='Rahul';
```

(j) Write a Query to delete the values in student table for rahul using **DELETE** command

```
mysql> delete from student where name='Rahul';
Query OK, 1 row affected (0.01 sec)
```

(k)Write a Query to **RENAME** table

```
mysql> rename table student to stu;
Query OK, 0 rows affected (0.26 sec)
```

(l) Write a Query to **DROP** the table;

```
mysql> drop table stu;
Query OK, 0 rows affected (0.06 sec)
```

Practical 21:MySQL queries using SINGLE FUNCTIONS(Numeric,String,Date)

Ans:

mysql>**CREATE TABLE emp (empno INT PRIMARY KEY,ename VARCHAR(30), sal DECIMAL(10,2), deptno INT);**

```
mysql> INSERT INTO emp VALUES
    -> (101, 'Ravi', 25000.75, 10),
    -> (102, 'Sneha', 32000.50, 20),
    -> (103, 'Arjun', 28000.00, 10),
    -> (104, 'Meena', 40000.25, 30),
    -> (105, 'Rahul', 15000.00, 20);
Query OK, 5 rows affected (0.03 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

NUMERIC FUNCTIONS:

Using POWER

```
mysql> select power(2,3) from dual;
```

Using ROUND

```
mysql> SELECT ROUND(2912.564, 1);
```

Using MOD

```
mysql> SELECT MOD(21, 2);
```

STRING FUNCTIONS

Using LOWER,UPPER

```
mysql> SELECT ename, UPPER(ename), LOWER(ename) FROM emp;
```

Using SUBSTR,LENGTH

```
mysql> SELECT ename, SUBSTRING(ename, 2, 3) ,length(ename) FROM emp;
```

Using LEFT,RIGHT

```
mysql> SELECT ename, LEFT(ename, 2), RIGHT(ename, 2) FROM emp;
```

Using TRIM,LTRIM,TRIM

```
mysql> SELECT ename, RTRIM(ename),LTRIM(ename),TRIM(ename) FROM emp;
```

Using INSTR

```
mysql> SELECT ename, INSTR(ename, 'a') FROM emp;
```

DATE FUNCTIONS

Using NOW()

```
mysql> SELECT NOW();
+---------------------+
| NOW()               |
+---------------------+
| 2025-10-12 06:51:49 |
+---------------------+
```

Using DATE()

```
mysql> SELECT DATE(NOW());
+--------------+
| DATE(NOW()) |
+--------------+
| 2025-10-12   |
```

Using MONTH()

```
mysql> SELECT MONTH(NOW());
+---------------+
| MONTH(NOW()) |
+---------------+
|            10 |
+---------------+
```

Using MONTHNAME

```
mysql> SELECT MONTHNAME('2025-10-12');
+-----------------------------+
| MONTHNAME('2025-10-12') |
+-----------------------------+
| October                     |
```

Using YEAR,DAY,DAYNAME

```
mysql> SELECT YEAR('2025-10-12'), DAY('2025-10-12'), DAYNAME('2025-10-12');
```

Practical 22: MySQL queries using AGGREGRATE FUNCTIONS(Max,Min,Sum,Avg,Count(*))

Ans: Create table STUD

mysql>>CREATE TABLE stud(rno INT PRIMARY KEY, name VARCHAR(50), class INT, eng INT, ip INT, mat INT, phy INT, che INT, total_marks INT, grade CHAR(2));

Insert Values in table

```
mysql> INSERT INTO stud VALUES
    -> (1, 'Ravi', 12, 88, 92, 85, 78, 90, 433, 'A'),
    -> (2, 'Sneha', 12, 95, 85, 92, 81, 76, 429, 'A'),
    -> (3, 'Arjun', 12, 80, 75, 68, 74, 70, 367, 'B'),
    -> (4, 'Meena', 12, 90, 96, 95, 89, 92, 462, 'A+'),
    -> (5, 'Rahul', 12, 58, 70, 55, 60, 65, 308, 'C');
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Use Aggregrate Functions from above table

Max:

```
mysql> SELECT MAX(mat) FROM stud;
```

Min :

```
mysql> SELECT MIN(phy) FROM stud;
```

Avg

```
mysql> SELECT AVG(ip) FROM stud;
```

Sum

```
mysql> SELECT SUM(total_marks) FROM stud;
```

Count (*)

```
mysql> SELECT COUNT(*) FROM stud;
```

Note :to use count function update the any value from table

```
mysql> UPDATE stud SET grade = NULL WHERE name = 'Rahul';
```

Count

```
mysql> SELECT COUNT(grade) FROM stud;
```

Practical 23: MySQL queries using GROUP BY,HAVING,ORDER BY(ASC,DESC)

Ans:

Ans: Create table STUD

```
mysql> CREATE TABLE stud(rno INT PRIMARY KEY,name VARCHAR(50),class INT,eng INT,ip INT,mat INT,phy INT, che INT, total_marks INT,
    ->     grade CHAR(2));
Query OK, 0 rows affected (0.04 sec)
```

Insert Values in table

```
mysql> INSERT INTO stud VALUES
    -> (1, 'Ravi', 12, 88, 92, 85, 78, 90, 433, 'A'),
    -> (2, 'Sneha', 12, 95, 85, 92, 81, 76, 429, 'A'),
    -> (3, 'Arjun', 12, 80, 75, 68, 74, 70, 367, 'B'),
    -> (4, 'Meena', 12, 90, 96, 95, 89, 92, 462, 'A+'),
    -> (5, 'Rahul', 12, 58, 70, 55, 60, 65, 308, 'C');
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Use GROUP BY,HAVING,ORDER BY(ASC,DESC)from above table

Group By:

```
mysql> SELECT class, AVG(ip) FROM stud GROUP BY class;
```

```
mysql> SELECT grade, COUNT(*) FROM stud GROUP BY grade;
```

```
mysql> SELECT grade, SUM(eng + ip + mat + phy + che) FROM stud GROUP BY grade;
```

Having:

```
mysql> SELECT class, AVG(total_marks) FROM stud GROUP BY class HAVING AVG(total_marks) > 300;
```

```
SELECT grade, COUNT(*) FROM stud GROUP BY grade HAVING COUNT(*) > 3;
```

```
mysql> SELECT class, MAX(total_marks) FROM stud GROUP BY class HAVING MAX(total_marks) > 450;
```

```
mysql> SELECT grade, AVG(mat) FROM stud GROUP BY grade HAVING AVG(mat) > 80;
```

Order By(ASC/DESC):

```
mysql> SELECT rno, name, class, total_marks FROM stud ORDER BY total_marks DESC;
```

```
mysql> SELECT rno, name, class, total_marks FROM stud ORDER BY total_marks ASC;
```

```
mysql> SELECT rno, name, class FROM stud ORDER BY name ASC;
```

```
mysql> SELECT rno, name, grade, total_marks FROM stud ORDER BY grade DESC;
```

Group By + Having + Order By

```
mysql> SELECT class, AVG(total_marks) FROM stud GROUP BY class HAVING AVG(total_marks) > 400
    -> ORDER BY AVG(total_marks) DESC;
```

```
mysql> SELECT grade, COUNT(*) FROM stud GROUP BY grade HAVING COUNT(*) > 2 ORDER BY grade ASC;
```

```
mysql> SELECT class, MAX(total_marks) FROM stud GROUP BY class HAVING MAX(total_marks) > 450 ORDER BY MAX(total_marks) DESC;
```

Practical 24:**MySQL queries using CARTESIAN PRODUCT or CROSS JOIN**

Ans:Create 2 tables

```
mysql> CREATE TABLE STU (StudentID INT PRIMARY KEY,Name VARCHAR(30), Class INT );
Query OK, 0 rows affected (0.27 sec)
```

```
mysql> INSERT INTO STU VALUES (1, 'Ankit', 12),(2, 'Priya', 11),(3, 'Rohan', 12);
```

```
mysql> CREATE TABLE SUBJECT (SubCode CHAR(2),    SubjectName VARCHAR(30));
```

```
mysql> INSERT INTO SUBJECT VALUES ('S1', 'IP'),('S2', 'Maths');
```

Using Cartesian Product

```
mysql> SELECT *  FROM STU, SUBJECT;
```

```
mysql> SELECT * FROM STU CROSS JOIN SUBJECT;
```

Cartesian Product with a Filtering Condition

```
mysql> SELECT Name, SubjectName FROM STU,SUBJECT WHERE STU.Class = 12;
```

```
mysql> SELECT STUD.Name, SUBJECT.SubjectName FROM STUD CROSS JOIN SUBJECT;
```

```
mysql> SELECT Name, SubjectName FROM STUD, SUBJECT WHERE SubjectName = 'Maths';
```

Practical 25:MySQL queries using EQUI JOIN & NATURAL JOIN

Ans:Create Two tables

```
mysql> CREATE TABLE student (admno INT PRIMARY KEY,name VARCHAR(30),class INT,  section CHAR(1));
```

```
mysql> CREATE TABLE marks (admno INT,subject VARCHAR(20), marks INT);
```

Insert Values

```
mysql> INSERT INTO student VALUES (101, 'Asha', 12, 'A'),(102, 'Raj', 12, 'B'),(103, 'Meena', 11, 'A'),(104, 'Rafiq', 12, 'A');
```

```
mysql> INSERT INTO marks VALUES(101, 'IP', 85),(102, 'IP', 78),(103, 'IP', 90),(104, 'IP', 88);
```

EQUI JOIN (using WHERE or INNER JOIN)

Display student name and marks using EQUI JOIN

```
mysql> SELECT name, marks
    -> FROM student, marks
    -> WHERE student.admno = marks.admno;
```

Display all data using EQUI JOIN

```
mysql> SELECT student.admno, name, class, section, subject, marks
    -> FROM student, marks
    -> WHERE student.admno = marks.admno;
```

Display students of class 12 with their marks

```
mysql> SELECT name, class, marks
    -> FROM student, marks
    -> WHERE student.admno = marks.admno AND class = 12;
```

Display students who scored more than 80

```
mysql> SELECT name, marks
    -> FROM student, marks
    -> WHERE student.admno = marks.admno AND marks > 80;
```

Display same query using INNER JOIN (alternative to EQUI JOIN)

```
mysql> SELECT student.admno, name, class, subject, marks
    -> FROM student INNER JOIN marks
    -> ON student.admno = marks.admno;
```

Display all data using NATURAL JOIN

```
mysql> SELECT * FROM student NATURAL JOIN marks;
```

Display name and marks of all students

```
mysql> SELECT name, marks FROM student NATURAL JOIN marks;
```

Display name, class, and marks of students of class 12

```
mysql> SELECT name, class, marks
    -> FROM student NATURAL JOIN marks
    -> WHERE class = 12;
```